

Interview Cram Sheet

Every lesson boiled down to the facts you need seconds before the test — pair it with the full study notes and the Drillbench quiz.

MODULE A · REACT

L1 · Introduction to React & JSX

- React = component-based UI; virtual DOM diffs old vs. new tree, patches only what changed (reconciliation).
- JSX compiles to `React.createElement()`; use `className`, camelCase events (`onClick`), `{}` for JS expressions.
- Components are PascalCase functions returning JSX; must return one root element (or a Fragment `<></>`).
- Props flow one direction: parent → child, read-only.

L2 · State: `useState` & `useEffect`

- `useState` returns `[value, setter]`; never mutate state directly, always call the setter.
- `useEffect` deps: `[]` = once on mount, `[x]` = reruns when `x` changes, no array = every render.
- Return a cleanup function from `useEffect` to cancel timers/subscriptions before rerun or unmount.
- Controlled input: value comes from state, `onChange` updates it.
- Pass the handler itself (`onClick={fn}`) — calling it (`onClick={fn()}`) runs it on every render instead.

L3 · React Router & Navigation

- `<Link>/<NavLink>` = client-side nav, no reload; `<a href>` = full reload, loses app state.
- `useParams()` reads `:id` from the URL; `useNavigate()` redirects programmatically.
- Nested routes render into `<Outlet />`; the catch-all `path="*" route` must be listed last.

L4 · Data Fetching & Context API

- Track loading / data / error as separate state when fetching inside `useEffect`.
- `Context = createContext + <Provider value=...> + useContext()` — avoids prop drilling.
- Don't use Context for fast-changing state: every consumer re-renders on any value change.

L7 · Advanced React — Custom Hooks & Performance

- Custom hook = a function named `use...` that calls other hooks; packages reusable stateful logic.
- `React.memo` skips a re-render when props are shallowly unchanged.
- `useMemo` caches a computed value; `useCallback` caches a function reference.
- `React.lazy` + `<Suspense>` = code splitting, load a component's code only when needed.
- Don't memoize cheap values — the bookkeeping can cost more than it saves.

MODULE B · TYPESCRIPT

L5 · TypeScript Basics & Setup

- `any` disables type checking entirely; `unknown` forces you to narrow before use — prefer `unknown`.
- `interface`: extendable/mergeable, best for object shapes; `type`: required for unions & intersections.
- Generics `<T>` keep full type info through a reusable function instead of falling back to `any`.

L6 · TypeScript with React

- `useState(null)` infers type `null` — annotate explicitly: `useState<User | null>(null)`.
- Event types: `React.ChangeEvent<HTMLInputElement>`, `React.MouseEvent<HTMLButtonElement>`.
- `children` is typed `React.ReactNode`.
- `React.FC` adds an implicit `children` prop and fights with generics — many teams type the function directly instead.

MODULE C · PYTHON

L8 · Python OOP — Classes, Inheritance, Encapsulation

- `__init__` = constructor; `self` = the instance, passed explicitly to every instance method.
- `@classmethod` takes `cls` (operates on the class); `@staticmethod` takes neither (just namespaced).
- Inheritance: `super().__init__()` calls the parent constructor; override methods for polymorphism (duck typing).
- `__name__` = convention only; `__name__` = name-mangled, NOT true privacy — Python has no enforced private.
- `__str__` = readable/user-facing; `__repr__` = unambiguous/debug-facing (also the `print()` fallback).

L9 · Python File Handling & Data Processing

- with `open(path)` as `f`: auto-closes the file even if an exception is raised inside the block.
- `csv.DictReader` for CSV rows as dicts; `json.load/json.dump` for JSON; `pathlib` for cross-platform paths.
- Catch specific exceptions (`FileNotFoundException`) — a bare `except`: hides real bugs too.

L10 · Python Web APIs with Flask

- `@app.route(path, methods=[...])` maps an HTTP verb + URL to a function.
- `request.args` = query params, `request.json` = JSON body, `request.form` = form-encoded data.
- Always return `jsonify(data)`, `status_code` — 200 read, 201 created, 400 bad input, 404 missing, 500 server error.

MODULE D · C# / .NET

L11 · C# Fundamentals & OOP

- `class` = reference type (shared, mutations visible everywhere); `struct` = value type (copied on assign/pass).
- `public` / `private` / `protected` control visibility (anywhere / class-only / class + subclasses).
- Naming: `PascalCase` for types & methods, `camelCase` for locals & parameters.

L12 · Control Flow & the Four Pillars of OOP

- `if/else if/switch` for branching; `for` / `foreach` / `while` / `do-while` (do-while always runs its body once first).
- Four pillars: encapsulation, inheritance, polymorphism, abstraction.
- `override` requires the base method to be `virtual` or `abstract` — otherwise it won't compile.
- `Overloading` = same name, different params, resolved at compile time (not the same as overriding).

L13 · Applying OOP — Student Management System pattern

- Keep data (a `Student` class) and operations (a `StudentManager` class) in separate classes.
- Route every `add/search/update` through manager methods, not scattered directly in `Main`.
- Use constructors to enforce a valid object from the moment it's created.

L14 · Advanced C# — LINQ, Collections, Exceptions

- LINQ has deferred execution: a query only runs when enumerated (`foreach` / `.ToList()`), not when declared.
- `Dictionary<K,V>` = lookup by key, `HashSet<T>` = uniqueness, `Queue<T>` = FIFO, `Stack<T>` = LIFO.
- `throw`; preserves the original stack trace; `throw ex`; resets it — prefer `throw`;
- Null-safe operators: `?.` (safe access), `??` (fallback), `??=` (assign only if null).

MODULE E · ASP.NET CORE

L15 · Introduction to .NET & ASP.NET Core

- Program.cs = entry point: registers DI services and builds the middleware pipeline.
- DI lifetimes: Transient (new every time), Scoped (one per request), Singleton (one for the app's life).
- Never inject a Scoped service (e.g. a DbContext) into a Singleton — causes stale/shared-state bugs.
- Middleware order matters — e.g. authentication must run before authorization.

L16 · ASP.NET Core & Web API Development

- [ApiController] / [Route] / [HttpGet] / [HttpPost] = attribute routing.
- [FromBody] is required to bind a JSON body to a complex parameter, or it silently comes back null.
- Match the response to reality: Ok() 200, CreatedAtAction() 201, NotFound() 404, BadRequest() 400.

L17 · Entity Framework Core & Database Integration

- DbContext = a session with the DB; DbSet<T> maps to a table.
- Nothing is persisted until SaveChanges() is called — Add()/Remove() only stage changes in memory.
- .Include(s => s.Courses) eager-loads related data, avoiding the N+1 query problem.
- dotnet ef migrations add generates a schema script; dotnet ef database update applies it.

L18 · API Authentication & Security with JWT

- Authentication = who you are; Authorization = what you're allowed to do.
- JWT = header.payload.signature, stateless, carries identity/role claims.
- [Authorize] protects an endpoint; [Authorize(Roles="Admin")] adds a role requirement.
- localStorage is readable by any JS on the page (XSS risk); an httpOnly cookie is not.

L19 · API Testing, Validation & Swagger

- Swagger/OpenAPI auto-generates interactive docs straight from your controllers/models.
- [Required]/[StringLength] + [ApiController] = automatic 400 response on an invalid model.
- Client-side validation is UX only — anyone can bypass it and call the API directly, so the server must re-validate everything.

MODULE F · DATABASES

L20 · SQL Server — Queries, Joins & Design

- INNER JOIN = matching rows only; LEFT JOIN = all of the left table + NULLs where there's no match.
- WHERE filters rows before grouping; HAVING filters groups after — aggregates only work in HAVING.
- Normalization (1NF/2NF/3NF) removes redundant, duplicated data.
- Primary key = unique row id; foreign key = enforces a valid reference; index = faster reads, slower writes.

L21 · NoSQL with MongoDB & Deployment

- SQL = fixed schema, strong joins & consistency; MongoDB = flexible-schema documents, scales out easily.
- MongoDB CRUD: insertOne / find / updateOne / deleteOne over documents (BSON) in a collection.
- React frontend → GitHub Pages; ASP.NET Core API → Azure App Service.
- CORS is enforced by the browser, not Postman — the API must explicitly allow the deployed frontend's origin.

Quick-Lookup Tables

HTTP status codes

Code	Meaning	Typical use
200 OK	Success	Successful GET / PUT / general read

Code	Meaning	Typical use
201 Created	Resource created	Successful POST that created something
400 Bad Request	Client sent invalid data	Failed validation
404 Not Found	No resource at that id/URL	GET/DELETE on a missing id
500 Server Error	Unhandled exception server-side	Bug, not the client's fault

SQL JOIN types

Join	Keeps	Unmatched side
INNER JOIN	Only rows matching on both sides	Dropped entirely
LEFT JOIN	All left rows + matches on the right	NULLs on the right
RIGHT JOIN	All right rows + matches on the left	NULLs on the left
FULL OUTER JOIN	All rows from both sides	NULLs on whichever side has no match

.NET dependency injection lifetimes

Lifetime	New instance	Good for
Transient	Every time it's requested	Lightweight, stateless services
Scoped	Once per HTTP request	DbContext, per-request state
Singleton	Once for the app's whole lifetime	Config, caches, stateless shared logic

React hooks, at a glance

Hook	Purpose
useState	Hold a value that triggers a re-render when it changes
useEffect	Run a side effect after render (fetch, subscribe, sync)
useContext	Read a value from the nearest matching Context Provider
useMemo	Cache an expensive computed value between renders
useCallback	Cache a function reference between renders